FILE WITH CODE FOR HOME WORK 3

```
# Read the data
data <- read.csv("data_for_analysis.csv", stringsAsFactors = FALSE)

# Identify hormone columns
hormone_cols <- c("hormone1", "hormone2", "hormone3", "hormone4",
                  "hormone5", "hormone6", "hormone7", "hormone8",
                  "hormone10_generated")

# Remove rows with missing values in hormones or outcome
complete_rows <- complete.cases(data[, c("outcome", hormone_cols)])
hormone_data <- data[complete_rows, ]

# Split by outcome group
group0_data <- hormone_data[hormone_data$outcome == 0, hormone_cols]
group1_data <- hormone_data[hormone_data$outcome == 1, hormone_cols]

# Function to calculate skewness (base R)
calc_skewness <- function(x) {
  x <- x[!is.na(x)]
  n <- length(x)
```

```
  if (n < 3) return(NA)
  mean_x <- mean(x)
  sd_x <- sd(x)
  if (sd_x == 0) return(NA)
  sum(((x - mean_x) / sd_x)^3) * n / ((n - 1) * (n - 2))
}

# Function to calculate kurtosis (base R)
calc_kurtosis <- function(x) {
  x <- x[!is.na(x)]
  n <- length(x)
  if (n < 4) return(NA)
  mean_x <- mean(x)
  sd_x <- sd(x)
  if (sd_x == 0) return(NA)
  sum(((x - mean_x) / sd_x)^4) * n * (n + 1) / ((n - 1) * (n - 2) * (n - 3)) -
    3 * (n - 1)^2 / ((n - 2) * (n - 3))
}

# 1. Descriptive statistics table
descriptive_stats <- function(data, group_name) {
  results <- data.frame(
    hormone = character(),
```

```
  n = numeric(),
  mean = numeric(),
  sd = numeric(),
  median = numeric(),
  min = numeric(),
  max = numeric(),
  skewness = numeric(),
  kurtosis = numeric(),
  group = character(),
  stringsAsFactors = FALSE
)

for (hormone in colnames(data)) {
  vals <- data[[hormone]]
  vals <- vals[!is.na(vals)]

  results <- rbind(results, data.frame(
    hormone = hormone,
    n = length(vals),
    mean = round(mean(vals), 4),
    sd = round(sd(vals), 4),
    median = round(median(vals), 4),
    min = round(min(vals), 4),
```

```
      max = round(max(vals), 4),
      skewness = round(calc_skewness(vals), 4),
      kurtosis = round(calc_kurtosis(vals), 4),
      group = group_name,
      stringsAsFactors = FALSE
    ))
  }
  return(results)
}

stats_group0 <- descriptive_stats(group0_data, "Outcome=0")
stats_group1 <- descriptive_stats(group1_data, "Outcome=1")

# Combine statistics
stats_combined <- rbind(stats_group0, stats_group1)
cat("\n========== DESCRIPTIVE STATISTICS ==========\n")
print(stats_combined)

# 2. Shapiro-Wilk Test (base R has shapiro.test)
# 3. Levene's Test (implement manually since car package not available)
levene_test_manual <- function(values, groups) {
  # Calculate medians for each group
  unique_groups <- unique(groups)
```

```
  group_medians <- sapply(unique_groups, function(g) median(values[groups == g]))

  # Calculate absolute deviations from median
  deviations <- abs(values - group_medians[match(groups, unique_groups)])

  # Perform ANOVA on deviations
  anova_result <- summary(aov(deviations ~ factor(groups)))
  p_value <- anova_result[[1]][["Pr(>F)"]][1]
  return(p_value)
}

# Perform tests for each hormone
test_results <- data.frame(
  hormone = character(),
  shapiro_group0_p = numeric(),
  shapiro_group1_p = numeric(),
  normal_group0 = character(),
  normal_group1 = character(),
  levene_p = numeric(),
  homogeneous = character(),
  stringsAsFactors = FALSE
)
```

```
for (hormone in hormone_cols) {
  # Extract values for each group
  group0_vals <- group0_data[[hormone]][!is.na(group0_data[[hormone]])]
  group1_vals <- group1_data[[hormone]][!is.na(group1_data[[hormone]])]

  # Shapiro-Wilk test for each group
  shapiro0 <- tryCatch(shapiro.test(group0_vals)$p.value, error = function(e) NA)
  shapiro1 <- tryCatch(shapiro.test(group1_vals)$p.value, error = function(e) NA)

  # Prepare data for Levene's test
  all_vals <- c(group0_vals, group1_vals)
  all_groups <- c(rep(0, length(group0_vals)), rep(1, length(group1_vals)))

  # Levene's test
  levene_p <- tryCatch(levene_test_manual(all_vals, all_groups), error = function(e)
NA)

  test_results <- rbind(test_results, data.frame(
    hormone = hormone,
    shapiro_group0_p = round(shapiro0, 4),
    shapiro_group1_p = round(shapiro1, 4),
    normal_group0 = ifelse(!is.na(shapiro0) && shapiro0 > 0.05, "Yes", "No"),
```

```
    normal_group1 = ifelse(!is.na(shapiro1) && shapiro1 > 0.05, "Yes", "No"),
    levene_p = round(levene_p, 4),
    homogeneous = ifelse(!is.na(levene_p) && levene_p > 0.05, "Yes", "No"),
    stringsAsFactors = FALSE
  ))
}

cat("\n========== NORMALITY AND HOMOGENEITY TESTS ==========\n")
print(test_results)

# 3. Q-Q Plots and Histograms
# Create directory
dir.create("hormone_plots", showWarnings = FALSE)

# Function for Q-Q plot
create_qq_plot <- function(vals, hormone, group_name) {
  vals <- vals[!is.na(vals)]
  if (length(vals) < 3) return()

  # Calculate theoretical quantiles
  n <- length(vals)
  theoretical <- qnorm((1:n - 0.5) / n)
```

```
  sample_quantiles <- sort(vals)

  plot(theoretical, sample_quantiles,
       main = paste(hormone, "-", group_name),
       xlab = "Theoretical Quantiles",
       ylab = "Sample Quantiles",
       pch = 16, col = "blue", cex = 0.8)

  # Add reference line (through 1st and 3rd quartiles)
  quantiles <- quantile(vals, c(0.25, 0.75), na.rm = TRUE)
  theo_quantiles <- qnorm(c(0.25, 0.75))
  slope <- diff(quantiles) / diff(theo_quantiles)
  intercept <- quantiles[1] - slope * theo_quantiles[1]
  abline(a = intercept, b = slope, col = "red", lwd = 2)
}

# Function for histogram
create_histogram <- function(vals, hormone, group_name) {
  vals <- vals[!is.na(vals)]
  if (length(vals) < 3) return()

  hist(vals,
       main = paste(hormone, "-", group_name),
```

```
      xlab = "Value",
      ylab = "Frequency",
      col = "steelblue",
      border = "black",
      breaks = "Sturges")

  # Add density line
  dens <- density(vals, na.rm = TRUE)
  lines(dens, col = "red", lwd = 2)
}

# Generate and save plots as PNG
for (hormone in hormone_cols) {
  group0_vals <- group0_data[[hormone]]
  group1_vals <- group1_data[[hormone]]

  # Histogram for Group 0
  png(paste0("hormone_plots/", hormone, "_group0_hist.png"), width = 600, height
= 500)
  create_histogram(group0_vals, hormone, "Outcome=0")
  dev.off()

  # Histogram for Group 1
```

```
  png(paste0("hormone_plots/", hormone, "_group1_hist.png"), width = 600, height
= 500)
  create_histogram(group1_vals, hormone, "Outcome=1")
  dev.off()

  # Q-Q plot for Group 0
  png(paste0("hormone_plots/", hormone, "_group0_qq.png"), width = 600, height
= 500)
  create_qq_plot(group0_vals, hormone, "Outcome=0")
  dev.off()

  # Q-Q plot for Group 1
  png(paste0("hormone_plots/", hormone, "_group1_qq.png"), width = 600, height
= 500)
  create_qq_plot(group1_vals, hormone, "Outcome=1")
  dev.off()
}

# Create combined PDF
pdf("hormone_plots/all_hormone_distributions.pdf", width = 12, height = 8)

for (hormone in hormone_cols) {
  group0_vals <- group0_data[[hormone]]
  group1_vals <- group1_data[[hormone]]
```

```
  # 2x2 layout
  par(mfrow = c(2, 2))

  # Histograms
  create_histogram(group0_vals, hormone, "Outcome=0")
  create_histogram(group1_vals, hormone, "Outcome=1")

  # Q-Q plots
  create_qq_plot(group0_vals, hormone, "Outcome=0")
  create_qq_plot(group1_vals, hormone, "Outcome=1")

  # Add overall title
  mtext(paste("Distribution of", hormone), side = 3, line = -2, outer = TRUE, cex =
1.5)
}

dev.off()

cat("\n========== PLOTS GENERATED ==========\n")
cat("Plots saved in 'hormone_plots' directory\n")

# 4. Statistical tests
```

```
# Brunner-Munzel test (implemented manually)
brunner_munzel_test_manual <- function(x, y) {
  x <- x[!is.na(x)]
  y <- y[!is.na(y)]
  nx <- length(x)
  ny <- length(y)
  n <- nx + ny

  # Create rank matrix
  all_vals <- c(x, y)
  ranks <- rank(all_vals)
  ranks_x <- ranks[1:nx]
  ranks_y <- ranks[(nx + 1):n]

  # Calculate rank means
  mean_rank_x <- mean(ranks_x)
  mean_rank_y <- mean(ranks_y)

  # Calculate variance
  s2_x <- var(ranks_x)
  s2_y <- var(ranks_y)

  # Brunner-Munzel statistic
```

```
  t_stat <- (mean_rank_x - mean_rank_y) / sqrt((s2_x / nx) + (s2_y / ny))

  # Degrees of freedom (Satterthwaite)
  df <- (s2_x / nx + s2_y / ny)^2 /
    ((s2_x / nx)^2 / (nx - 1) + (s2_y / ny)^2 / (ny - 1))

  # P-value
  p_value <- 2 * pt(-abs(t_stat), df)
  return(p_value)
}

# Perform all tests
test_comparison <- data.frame(
  hormone = character(),
  t_test_p = numeric(),
  welch_p = numeric(),
  wilcox_p = numeric(),
  brunner_munzel_p = numeric(),
  recommendation = character(),
  stringsAsFactors = FALSE
)

for (hormone in hormone_cols) {
```

```
# Extract values for each group
group0_vals <- group0_data[[hormone]][!is.na(group0_data[[hormone]])]
group1_vals <- group1_data[[hormone]][!is.na(group1_data[[hormone]])]

# Skip if insufficient data
if (length(group0_vals) < 3 | length(group1_vals) < 3) {
  test_comparison <- rbind(test_comparison, data.frame(
    hormone = hormone,
    t_test_p = NA,
    welch_p = NA,
    wilcox_p = NA,
    brunner_munzel_p = NA,
    recommendation = "Insufficient data",
    stringsAsFactors = FALSE
  ))
  next
}

# t-test (assuming equal variance)
t_test <- tryCatch(t.test(group0_vals, group1_vals, var.equal = TRUE)$p.value,
                   error = function(e) NA)

# Welch's t-test (not assuming equal variance)
```

```
  welch_test <- tryCatch(t.test(group0_vals, group1_vals, var.equal = FALSE)$p.value,
                         error = function(e) NA)

  # Wilcoxon rank-sum test
  wilcox_test <- tryCatch(wilcox.test(group0_vals, group1_vals)$p.value,
                          error = function(e) NA)

  # Brunner-Munzel test
  bm_test <- tryCatch(brunner_munzel_test_manual(group0_vals, group1_vals),
                      error = function(e) NA)

  # Get normality and homogeneity info
  norm0 <- test_results[test_results$hormone == hormone, "normal_group0"]
  norm1 <- test_results[test_results$hormone == hormone, "normal_group1"]
  homog <- test_results[test_results$hormone == hormone, "homogeneous"]

  # Determine recommendation
  if (norm0 == "Yes" && norm1 == "Yes" && homog == "Yes" && !is.na(t_test)) {
    recommendation <- "t-test (parametric, equal variance)"
  } else if (norm0 == "Yes" && norm1 == "Yes" && homog == "No" && !is.na(welch_test)) {
    recommendation <- "Welch's t-test (parametric, unequal variance)"
```

```
  } else {
    recommendation <- "Brunner-Munzel (non-parametric, robust)"
  }

  test_comparison <- rbind(test_comparison, data.frame(
    hormone = hormone,
    t_test_p = round(t_test, 4),
    welch_p = round(welch_test, 4),
    wilcox_p = round(wilcox_test, 4),
    brunner_munzel_p = round(bm_test, 4),
    recommendation = recommendation,
    stringsAsFactors = FALSE
  ))
}

cat("\n========== STATISTICAL TEST COMPARISONS ==========\n")
print(test_comparison)

# 5. Correlation heatmaps (using base R heatmap function)

# Determine correlation method based on normality
normal_count_group0 <- sum(test_results$normal_group0 == "Yes", na.rm = TRUE)
```

```
normal_count_group1 <- sum(test_results$normal_group1 == "Yes", na.rm = TRUE)

total_hormones <- length(hormone_cols)


cor_method_group0 <- ifelse(normal_count_group0 > total_hormones/2, "pearson", "spearman")

cor_method_group1 <- ifelse(normal_count_group1 > total_hormones/2, "pearson", "spearman")


# Function to calculate correlation matrix

calc_cor_matrix <- function(data, method) {

  n_vars <- ncol(data)

  cor_matrix <- matrix(1, nrow = n_vars, ncol = n_vars)

  colnames(cor_matrix) <- colnames(data)

  rownames(cor_matrix) <- colnames(data)


  for (i in 1:n_vars) {

    for (j in 1:n_vars) {

      if (i != j) {

        x <- data[[i]][!is.na(data[[i]])]

        y <- data[[j]][!is.na(data[[j]])]


        # Only use complete pairs

        complete_idx <- complete.cases(data[, c(i, j)])
```

```
      x <- data[complete_idx, i]
      y <- data[complete_idx, j]

      if (length(x) > 2) {
        if (method == "pearson") {
          cor_matrix[i, j] <- cor(x, y, method = "pearson")
        } else {
          # Spearman correlation
          cor_matrix[i, j] <- cor(rank(x), rank(y), method = "pearson")
        }
      }
    }
  }
  return(cor_matrix)
}

# Calculate correlation matrices
cor_matrix_group0 <- calc_cor_matrix(group0_data, cor_method_group0)
cor_matrix_group1 <- calc_cor_matrix(group1_data, cor_method_group1)

cat("\n========== CORRELATION MATRICES ==========\n")
```

```
cat("\nCorrelation Matrix for Group 0 (Outcome=0) - Method:", toupper(cor_method_group0), "\n")
print(round(cor_matrix_group0, 3))

cat("\nCorrelation Matrix for Group 1 (Outcome=1) - Method:", toupper(cor_method_group1), "\n")
print(round(cor_matrix_group1, 3))

# Function to create heatmap using base R
create_heatmap_base <- function(cor_matrix, title, filename) {
  png(filename, width = 800, height = 800)

  # Create color palette
  colors <- colorRampPalette(c("blue", "white", "red"))(100)

  # Create heatmap
  heatmap(cor_matrix,
          col = colors,
          symm = TRUE,
          main = title,
          margins = c(8, 8),
          cexCol = 0.8,
          cexRow = 0.8,
```

```
        key = TRUE,
        key.title = "Correlation",
        key.xlab = "Correlation Coefficient",
        trace = "none")

  dev.off()
}

# Create heatmaps
create_heatmap_base(cor_matrix_group0,
                    paste("Correlation Heatmap - Outcome=0\nMethod:", toupper(cor_method_group0)),
                    "hormone_plots/correlation_heatmap_group0.png")

create_heatmap_base(cor_matrix_group1,
                    paste("Correlation Heatmap - Outcome=1\nMethod:", toupper(cor_method_group1)),
                    "hormone_plots/correlation_heatmap_group1.png")

# Create a simplified correlation plot using image()
png("hormone_plots/correlation_simple_group0.png", width = 800, height = 800)
image(x = 1:ncol(cor_matrix_group0), y = 1:ncol(cor_matrix_group0),
      z = cor_matrix_group0,
```

```
    col = colorRampPalette(c("blue", "white", "red"))(100),
    axes = FALSE,
    main = paste("Correlation - Group 0 (", toupper(cor_method_group0), ")", sep = ""),
    xlab = "", ylab = "")
axis(1, at = 1:ncol(cor_matrix_group0), labels = colnames(cor_matrix_group0), las = 2, cex.axis = 0.7)
axis(2, at = 1:ncol(cor_matrix_group0), labels = rownames(cor_matrix_group0), las = 2, cex.axis = 0.7)
# Add correlation values
for (i in 1:ncol(cor_matrix_group0)) {
  for (j in 1:ncol(cor_matrix_group0)) {
    text(i, j, round(cor_matrix_group0[i, j], 2), cex = 0.6)
  }
}
dev.off()


png("hormone_plots/correlation_simple_group1.png", width = 800, height = 800)
image(x = 1:ncol(cor_matrix_group1), y = 1:ncol(cor_matrix_group1),
    z = cor_matrix_group1,
    col = colorRampPalette(c("blue", "white", "red"))(100),
    axes = FALSE,
    main = paste("Correlation - Group 1 (", toupper(cor_method_group1), ")", sep = ""),
```

```
     xlab = "", ylab = "")

axis(1, at = 1:ncol(cor_matrix_group1), labels = colnames(cor_matrix_group1), las = 2, cex.axis = 0.7)

axis(2, at = 1:ncol(cor_matrix_group1), labels = rownames(cor_matrix_group1), las = 2, cex.axis = 0.7)

for (i in 1:ncol(cor_matrix_group1)) {

  for (j in 1:ncol(cor_matrix_group1)) {

    text(i, j, round(cor_matrix_group1[i, j], 2), cex = 0.6)

  }

}

dev.off()


# Export all results to CSV

# Add significance indicators to test results

test_results$significant_shapiro0 <- ifelse(test_results$shapiro_group0_p < 0.05, "Significant (non-normal)", "Not significant (normal)")

test_results$significant_shapiro1 <- ifelse(test_results$shapiro_group1_p < 0.05, "Significant (non-normal)", "Not significant (normal)")

test_results$significant_levene <- ifelse(test_results$levene_p < 0.05, "Significant (unequal variances)", "Not significant (equal variances)")


# Add significance to test comparisons

test_comparison$significant_bm <- ifelse(test_comparison$brunner_munzel_p < 0.05, "Yes (p < 0.05)", "No")
```

```
test_comparison$significant_wilcox <- ifelse(test_comparison$wilcox_p < 0.05, "Yes (p < 0.05)", "No")

test_comparison$significant_ttest <- ifelse(test_comparison$t_test_p < 0.05, "Yes (p < 0.05)", "No")


# Save all results

write.csv(stats_combined, "descriptive_statistics.csv", row.names = FALSE)

write.csv(test_results, "normality_tests.csv", row.names = FALSE)

write.csv(test_comparison, "group_comparisons.csv", row.names = FALSE)


# Create correlation matrix outputs

cor_matrix_group0_df <- as.data.frame(cor_matrix_group0)

cor_matrix_group0_df$variable <- rownames(cor_matrix_group0)

cor_matrix_group0_df <- cor_matrix_group0_df[, c(ncol(cor_matrix_group0_df), 1:(ncol(cor_matrix_group0_df)-1))]

write.csv(cor_matrix_group0_df, "correlation_matrix_group0.csv", row.names = FALSE)


cor_matrix_group1_df <- as.data.frame(cor_matrix_group1)

cor_matrix_group1_df$variable <- rownames(cor_matrix_group1)

cor_matrix_group1_df <- cor_matrix_group1_df[, c(ncol(cor_matrix_group1_df), 1:(ncol(cor_matrix_group1_df)-1))]

write.csv(cor_matrix_group1_df, "correlation_matrix_group1.csv", row.names = FALSE)
```

```
# Create text summary
sink("hormone_analysis_summary.txt")

cat("============================================\n")
cat("HORMONE ANALYSIS SUMMARY REPORT\n")
cat("============================================\n\n")

cat("1. DATASET OVERVIEW\n")
cat("------------------\n")
cat("Total number of observations:", nrow(hormone_data), "\n")
cat("Group 0 (Outcome=0) size:", nrow(group0_data), "\n")
cat("Group 1 (Outcome=1) size:", nrow(group1_data), "\n")
cat("Number of hormones analyzed:", length(hormone_cols), "\n\n")

cat("2. NORMALITY ASSESSMENT (Shapiro-Wilk test, alpha = 0.05)\n")
cat("---------------------------------------------------\n")
cat("Hormones normal in Group 0:\n")
cat(paste(test_results$hormone[test_results$normal_group0 == "Yes"], collapse =
", "))
cat("\n\nHormones normal in Group 1:\n")
cat(paste(test_results$hormone[test_results$normal_group1 == "Yes"], collapse =
", "))
cat("\n\n")
```

```
cat("3. HOMOGENEITY OF VARIANCE (Levene's test, alpha = 0.05)\n")

cat("-----------------------------------------------------\n")

cat("Hormones with equal variances:\n")

cat(paste(test_results$hormone[test_results$homogeneous == "Yes"], collapse = ", "))

cat("\n\n")


cat("4. RECOMMENDED STATISTICAL TESTS\n")

cat("------------------------------\n")

for (i in 1:nrow(test_comparison)) {

  cat(test_comparison$hormone[i], ": ", test_comparison$recommendation[i], "\n", sep = "")

}

cat("\n")


cat("5. SIGNIFICANT DIFFERENCES (Brunner-Munzel test, p < 0.05)\n")

cat("-----------------------------------------------------------\n")

sig_hormones <- test_comparison$hormone[!is.na(test_comparison$brunner_munzel_p) & test_comparison$brunner_munzel_p < 0.05]

if (length(sig_hormones) > 0) {

  for (h in sig_hormones) {

    p_val <- test_comparison$brunner_munzel_p[test_comparison$hormone == h]
```

```
    cat(h, ": p =", p_val, "\n")
  }
} else {
  cat("No hormones showed significant differences at alpha = 0.05\n")
}
cat("\n")

cat("6. CORRELATION ANALYSIS\n")
cat("----------------------\n")
cat("Correlation method used for Group 0:", toupper(cor_method_group0), "\n")
cat("Correlation method used for Group 1:", toupper(cor_method_group1), "\n")
cat("\nReasoning: Method selected based on normality of data\n")
cat("- Pearson: assumes normal distribution (used if >50% of hormones are normal)\n")
cat("- Spearman: non-parametric (used if ≤50% of hormones are normal)\n\n")

cat("7. STRONGEST CORRELATIONS IN GROUP 0 (|r| > 0.6)\n")
cat("-----------------------------------------------\n")
for (i in 1:ncol(cor_matrix_group0)) {
  for (j in 1:ncol(cor_matrix_group0)) {
    if (i < j && abs(cor_matrix_group0[i, j]) > 0.6) {
      cat(colnames(cor_matrix_group0)[i], "vs", colnames(cor_matrix_group0)[j],
          ": r =", round(cor_matrix_group0[i, j], 3), "\n")
```

```
    }
  }
}
cat("\n")

cat("8. STRONGEST CORRELATIONS IN GROUP 1 (|r| > 0.6)\n")
cat("-----------------------------------------------\n")
for (i in 1:ncol(cor_matrix_group1)) {
  for (j in 1:ncol(cor_matrix_group1)) {
    if (i < j && abs(cor_matrix_group1[i, j]) > 0.6) {
      cat(colnames(cor_matrix_group1)[i], "vs", colnames(cor_matrix_group1)[j],
          ": r =", round(cor_matrix_group1[i, j], 3), "\n")
    }
  }
}

sink()
```